

censtrunc

Censored and Truncated Normal Regression with Arbitrary Thresholds

Author: <Your Name>

Package: `censtrunc` v0.1.0

Abstract

This report documents `censtrunc`, a Python package for maximum-likelihood estimation of censored and truncated normal regression models with **arbitrary left and right thresholds**. The package generalises the classical Type-1 Tobit model (Tobin, 1958) and the truncated normal regression to two-sided limits, while exposing a scikit-learn-style API and a statsmodels-style results summary. Estimation uses Olsen's (1978) globally concave reparameterisation for numerical reliability. The report presents the statistical model, then three worked examples: two-sided censoring on simulated data, the classical left-censored Tobit on the Fair (1978) affairs dataset, and a truncated regression.

1. Statistical Model

1.1 Latent regression

All models share the latent linear specification with normal errors:

$$Y^* = X'\beta + e, \quad e \mid X \sim \mathcal{N}(0, \sigma^2).$$

What differs is the rule linking the latent Y^* to the observed data.

1.2 Censored model

Values outside $[L, R]$ are **clipped** to the nearest threshold:

$$Y = \min(R, \max(L, Y^*)).$$

The log-likelihood combines the probability mass at each threshold with the normal density on the interior. With $\alpha_L = (L - X'\beta)/\sigma$ and $\alpha_R = (R - X'\beta)/\sigma$,

$$\ell(\beta, \sigma) = \sum_{Y_i=L} \log \Phi(\alpha_{L,i}) + \sum_{Y_i=R} \log [1 - \Phi(\alpha_{R,i})] + \sum_{L < Y_i < R} [\log \phi(\alpha_i) - \log \sigma],$$

where ϕ, Φ are the standard normal pdf and cdf. The classical Tobit is the special case $L = 0, R = +\infty$.

1.3 Olsen's reparameterisation

Optimisation is carried out in $(\gamma, \nu) = (\beta/\sigma, 1/\sigma)$. Olsen (1978) showed the resulting log-likelihood is **globally concave**, so any gradient-based optimiser converges to the unique maximum. After convergence the package back-transforms to (β, σ) and computes standard errors from the observed information matrix.

1.4 Truncated model

In a truncated sample, observations outside (L, R) are **absent** entirely. The density is the normal density renormalised by the probability of falling inside the interval:

$$f(y_i | L < Y_i^* < R) = \frac{\sigma^{-1} \phi((y_i - X_i' \beta)/\sigma)}{\Phi(\alpha_{R,i}) - \Phi(\alpha_{L,i})}.$$

1.5 Conditional means and marginal effects

Three conditional means are reported (Hansen, 2022, ch. 27):

- **Latent:** $\mathbb{E}[Y^* | X] = X' \beta$
- **Censored:** $\mathbb{E}[Y | X]$, accounting for the threshold mass
- **Truncated:** $\mathbb{E}[Y | X, L < Y < R]$

Marginal effects are available as **AME** (average over the sample) and **MEM** (evaluated at the mean regressor), each with delta-method standard errors. The censored marginal effect on $\mathbb{E}[Y | X]$ equals $\beta_j \cdot [\Phi(\alpha_R) - \Phi(\alpha_L)]$ — the latent slope shrunk by the probability of being interior.

1.6 Inference

Every fitted model carries an overall likelihood-ratio test against the intercept-only null model and McFadden's pseudo- R^2 . Arbitrary nested models can be compared with `lr_test`, which returns $LR = 2(\ell_{\text{full}} - \ell_{\text{restricted}}) \sim \chi_q^2$.

2. Worked Examples

The remainder of this report reproduces three example notebooks. Each is fully executed; the code and its output appear inline.

2.1 Example 1 — Two-Sided Censored Regression

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from censtrunc import CensoredRegression, lr_test

rng = np.random.default_rng(2026)
n = 3000
```

Data-generating process

$$Y_i^* = 1.0 + 0.5 X_{1i} - 0.3 X_{2i} + 0.2 X_{3i} + e_i, \quad e_i \sim \mathcal{N}(0, 1).$$

The observation rule clips values below $L = 0$ and above $R = 2.5$, producing a realistic two-sided censoring pattern (think 'satisfaction score on a 0-2.5 scale').

```
In [2]: X = rng.normal(size=(n, 3))
beta_true = np.array([1.0, 0.5, -0.3, 0.2])
sigma_true = 1.0
y_star = beta_true[0] + X @ beta_true[1:] + rng.normal(scale=sigma_true, size=n)
L, R = 0.0, 2.5
y = np.clip(y_star, L, R)

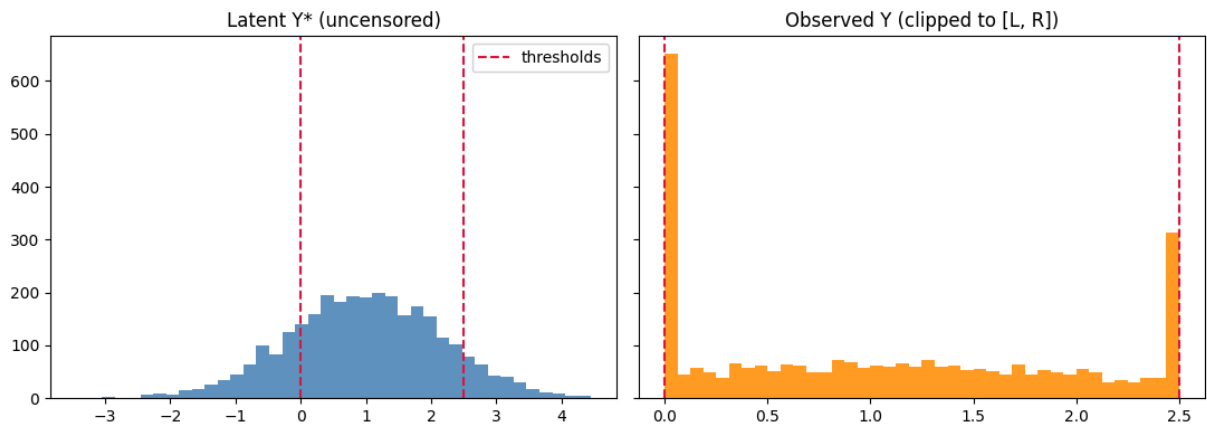
print(f'Left-censored:  {np.mean(y == L):.1%}')
print(f'Right-censored: {np.mean(y == R):.1%}')
print(f'Interior:       {np.mean((y > L) & (y < R)):.1%}')
```

```
Left-censored:  20.0%
Right-censored:  9.8%
Interior:       70.2%
```

Distribution of the latent and observed variables

The pile-ups at L and R in the observed distribution are the hallmark of two-sided censoring.

```
In [3]: fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
axes[0].hist(y_star, bins=40, color='steelblue', alpha=0.85)
axes[0].set_title('Latent Y* (uncensored)')
axes[0].axvline(L, color='crimson', linestyle='--', label='thresholds')
axes[0].axvline(R, color='crimson', linestyle='--')
axes[0].legend()
axes[1].hist(y, bins=40, color='darkorange', alpha=0.85)
axes[1].set_title('Observed Y (clipped to [L, R])')
axes[1].axvline(L, color='crimson', linestyle='--')
axes[1].axvline(R, color='crimson', linestyle='--')
plt.tight_layout(); plt.show()
```



Fit the censored regression

The package's default optimisation uses Olsen's reparameterisation, which makes the log-likelihood globally concave and guarantees convergence of gradient-based optimisers (Olsen 1978).

```
In [4]: model = CensoredRegression(left=L, right=R).fit(
        X, y, feature_names=['x1', 'x2', 'x3']
        )
print(model.summary())
```

Censored Regression Results						
Dep. Variable:	y	No. Observations:	3000			
Model:	CensoredRegression	Df Model:	4			
Method:	MLE	Df Residuals:	2995			
Date:	Sun, 31 May 2026	Log-Likelihood:	-3831.9928			
Time:	19:47:16	LL-Null:	-4286.2006			
AIC:	7673.9855	LLR p-value:	1.323e-196			
BIC:	7704.0174	Pseudo R-squ.:	0.1060			
Left threshold:	0	Right threshold:	2.5			
	coef	std err	z	P> z	[0.025	0.975]
sigma	0.9959	0.0165	60.3216	0.0000	0.9635	1.0282
const	0.9963	0.0190	52.4873	0.0000	0.9591	1.0336
x1	0.5126	0.0195	26.2589	0.0000	0.4744	0.5509
x2	-0.2565	0.0188	-13.6315	0.0000	-0.2934	-0.2196
x3	0.2030	0.0192	10.5893	0.0000	0.1655	0.2406
Left-censored observations: 601 (20.03%)						
Right-censored observations: 293 (9.77%)						
Uncensored observations: 2106 (70.20%)						
Optimiser converged: True						

Compare against OLS

OLS on the censored sample is biased (Greene 1981): the slope coefficients are shrunk roughly in proportion to the censoring probability. Our Tobit-style MLE is consistent.

```
In [5]: X_with_int = np.column_stack([np.ones(n), X])
ols_beta, *_ = np.linalg.lstsq(X_with_int, y, rcond=None)

comparison = pd.DataFrame({
    'true': beta_true,
    'OLS (biased)': ols_beta,
```

```
'Censored MLE': model.coef_,
}, index=['const', 'x1', 'x2', 'x3'])
comparison['OLS bias'] = comparison['OLS (biased)'] - comparison['true']
comparison['MLE bias'] = comparison['Censored MLE'] - comparison['true']
comparison.round(4)
```

Out [5]:

	true	OLS (biased)	Censored MLE	OLS bias	MLE bias
const	1.0	1.0706	0.9963	0.0706	-0.0037
x1	0.5	0.3648	0.5126	-0.1352	0.0126
x2	-0.3	-0.1794	-0.2565	0.1206	0.0435
x3	0.2	0.1460	0.2030	-0.0540	0.0030

Predictions: six quantities, one call

`model.predict(X)` returns a dataframe with all six quantities — three conditional means and three region probabilities. Each column has a one-letter mnemonic:

- **h** — *hidden* / latent mean $\mathbb{E}[Y^* | X] = X'\beta$
- **c** — *censored* mean $\mathbb{E}[Y | X]$
- **t** — *truncated* mean $\mathbb{E}[Y | X, L < Y < R]$
- **l** — left-region probability $P(Y = L | X) = \Phi(\alpha_L)$
- **m** — middle probability $P(L < Y < R | X) = \Phi(\alpha_R) - \Phi(\alpha_L)$
- **r** — right-region probability $P(Y = R | X) = 1 - \Phi(\alpha_R)$

In [6]:

```
X_show = X[:6]
model.predict(X_show).round(3) # default = all six columns
```

Out [6]:

	latent	censored	truncated	prob_left	prob_interior	prob_right
0	0.143	0.470	0.816	0.443	0.548	0.009
1	1.489	1.438	1.351	0.067	0.778	0.155
2	0.704	0.831	1.023	0.240	0.725	0.036
3	0.800	0.901	1.062	0.211	0.745	0.044
4	1.018	1.068	1.152	0.153	0.778	0.068
5	0.896	0.974	1.101	0.184	0.762	0.054

Subsets are selected with the same letter codes. For example, just the three probabilities (note that each row sums to one) or just the hidden mean (a 1-D array):

In [7]:

```
model.predict(X_show, kind='lmr').round(3)
```

```
Out [7]:
```

	prob_left	prob_interior	prob_right
0	0.443	0.548	0.009
1	0.067	0.778	0.155
2	0.240	0.725	0.036
3	0.211	0.745	0.044
4	0.153	0.778	0.068
5	0.184	0.762	0.054

```
In [8]: model.predict(X_show, kind='h') # single letter -> 1-D ndarray
```

```
Out [8]: array([0.14305362, 1.48881203, 0.70415823, 0.80034359, 1.01807358,
0.89647868])
```

Long names — `kind='latent'`, `'censored'`, `'truncated'` — remain valid and return 1-D arrays for backward compatibility.

Marginal effects via `get_margeff`

The marginal-effects API mirrors statsmodels' `get_margeff`: one method, with options for *where* to evaluate (`at`) and *what* to report (`method`). The summary prints in the familiar statsmodels layout.

```
In [9]: # AME = average over the sample (at='overall'); statsmodels-style summary
print(model.get_margeff(at='overall', method='dydx', kind='censored').summary())
```

```

Censored Regression Marginal Effects
=====
Dep. Variable:          y
Method:             dydx
At:                overall
=====

```

	dy/dx	std err	z	P> z	[0.025	0.975]
x1	0.3600	0.012	30.227	0.000	0.337	0.383
x2	-0.1801	0.013	-14.113	0.000	-0.205	-0.155
x3	0.1426	0.013	10.805	0.000	0.117	0.168

```
=====
```

`at='overall'` is the AME; `at='mean'` is the marginal effect at the mean (MEM). Below we compare the censored marginal effect across evaluation points and against the latent effect (which equals β exactly).

```
In [10]: import pandas as pd
summary = pd.DataFrame({
    'latent (=beta)': model.get_margeff(at='overall', kind='latent').margeff,
    'censored AME': model.get_margeff(at='overall', kind='censored').margeff,
    'censored MEM': model.get_margeff(at='mean', kind='censored').margeff,
    'truncated AME': model.get_margeff(at='overall', kind='truncated').margeff,
}, index=['x1', 'x2', 'x3'])
summary.round(4)
```

```
Out[10]:
```

	latent (=beta)	censored AME	censored MEM	truncated AME
x1	0.5126	0.3600	0.3973	0.2005
x2	-0.2565	-0.1801	-0.1988	-0.1003
x3	0.2030	0.1426	0.1573	0.0794

`method` switches between the derivative and elasticities: `dydx` (derivative), `eyex` (elasticity), `dyex / eydx` (semi-elasticities). Discrete regressors can be handled with `dummy=True` (a 0->1 difference rather than a derivative).

```
In [11]: elas = model.get_margeff(at='overall', method='eyex', kind='censored')
         elas.summary_frame().round(4)
```

```
Out[11]:
```

	eyex	std err	z	P> z	[0.025	0.975]
x1	-0.1238	0.0093	-13.3389	0.0	-0.1420	-0.1056
x2	-0.0317	0.0045	-7.0004	0.0	-0.0406	-0.0228
x3	-0.0193	0.0035	-5.4717	0.0	-0.0263	-0.0124

Marginal effects on the region probabilities

Beyond the conditional mean, we can ask how a regressor shifts the *probability* of each region: landing at the left threshold ($P(Y = L) = \Phi(\alpha_L)$), in the interior ($P(L < Y < R) = \Phi(\alpha_R) - \Phi(\alpha_L)$), or at the right threshold ($P(Y = R) = 1 - \Phi(\alpha_R)$). These are selected with `kind='prob-left'`, `'prob-interior'`, `'prob-right'`. Because the three probabilities sum to one, their marginal effects sum to zero.

```
In [12]: prob_effects = pd.DataFrame({
         'P(Y=L)': model.get_margeff(kind='prob-left').margeff,
         'P(L<Y<R)': model.get_margeff(kind='prob-interior').margeff,
         'P(Y=R)': model.get_margeff(kind='prob-right').margeff,
         }, index=['x1', 'x2', 'x3'])
         prob_effects['sum (=0)'] = prob_effects.sum(axis=1)
         prob_effects.round(4)
```

```
Out[12]:
```

	P(Y=L)	P(L<Y<R)	P(Y=R)	sum (=0)
x1	-0.1221	0.0461	0.0760	0.0
x2	0.0611	-0.0231	-0.0380	0.0
x3	-0.0484	0.0183	0.0301	0.0

A positive coefficient pushes observations out of the left pile-up and toward the right one, so the `P(Y=L)` effect is negative and the `P(Y=R)` effect is positive; the `sum (=0)` column confirms the adding-up constraint holds numerically.

Likelihood-ratio test

We test whether `x3` is jointly redundant by fitting a restricted model that omits it, then comparing log-likelihoods.

```
In [13]: restricted = CensoredRegression(left=L, right=R).fit(X[:, :2], y)
res = lr_test(model, restricted)
print(res)
```

```
Likelihood-ratio test
LR statistic : 111.3544
df           : 1
p-value      : 4.949e-26
log-lik full : -3831.9928
log-lik rest.: -3887.6700
n            : 3000
```

If the p-value is small, we reject the restriction (drop `x3`). Here `x3` has a true coefficient of 0.2 and a large sample, so the test correctly rejects.

LR test from a hypothesis string

Re-fitting a restricted model by hand is fine for simple drop-a-column cases, but cumbersome for joint or composite restrictions. For those, `model.lr_test(hypotheses)` mirrors statsmodels' `f_test` and accepts linear restrictions directly as a string — single, joint, or with arithmetic on parameter names.

```
In [14]: # Drop a single regressor (same null as the two-model test above)
print(model.lr_test('x3 = 0'))
```

```
Likelihood-ratio test
LR statistic : 111.3544
df           : 1
p-value      : 4.949e-26
log-lik full : -3831.9928
log-lik rest.: -3887.6700
n            : 3000
```

```
In [15]: # Joint restriction: x2 AND x3 are zero (composite null)
print(model.lr_test('(x2 = 0), (x3 = 0)'))
```

```
Likelihood-ratio test
LR statistic : 286.4290
df           : 2
p-value      : 6.35e-63
log-lik full : -3831.9928
log-lik rest.: -3975.2073
n            : 3000
```

```
In [16]: # Composite restriction with arithmetic: x1 - 2*x2 = 0
# (equivalent to testing whether x1 equals twice x2)
print(model.lr_test('x1 - 2*x2 = 0'))
```

```
Likelihood-ratio test
LR statistic : 549.6200
df           : 1
p-value      : 1.523e-121
log-lik full : -3831.9928
log-lik rest.: -4106.8028
n            : 3000
```


Each call refits the model under the linear constraint $R\hat{\theta} = r$ (via `scipy.optimize` with `LinearConstraint`) and reports the asymptotic LR statistic $2(\ell_{\text{full}} - \ell_{\text{rest.}}) \sim \chi_q^2$.

2.2 Example 2 — Classical Tobit (Fair Affairs Data)

```
In [17]: import numpy as np
import pandas as pd
import statsmodels.api as sm
from cens trunc import CensoredRegression

data = sm.datasets.fair.load_pandas().data
print('n =', len(data))
print(f'share with affairs == 0: {(data['affairs'] == 0).mean():.1%}')
data.head()
```

```
n = 6366
share with affairs == 0: 67.8%
```

```
Out[17]:
```

	rate_marriage	age	yrs_married	children	religious	educ	occupation	occupation
0	3.0	32.0	9.0	3.0	3.0	17.0	2.0	
1	3.0	27.0	13.0	3.0	1.0	14.0	3.0	
2	4.0	22.0	2.5	0.0	1.0	16.0	3.0	
3	4.0	37.0	16.5	4.0	3.0	16.0	5.0	
4	5.0	27.0	9.0	1.0	1.0	14.0	3.0	

Specification

Following Fair (1978), we regress the count of affairs on the standard set of demographic and marital variables.

```
In [18]: covariates = ['rate_marriage', 'age', 'yrs_married', 'children',
                    'religious', 'educ', 'occupation', 'occupation_husb']
X = data[covariates]
y = data['affairs']
```

Fit

We tell `CensoredRegression` that the left threshold is zero; the right threshold is left unspecified.

```
In [19]: model = CensoredRegression(left=0.0).fit(X, y)
print(model.summary())
```

Censored Regression Results						
Dep. Variable:	y	No. Observations:	6366			
Model:	CensoredRegression	Df Model:	9			
Method:	MLE	Df Residuals:	6356			
Date:	Sun, 31 May 2026	Log-Likelihood:	-7804.3803			
Time:	19:47:16	LL-Null:	-8155.0240			
AIC:	15628.7605	LLR p-value:	3.780e-146			
BIC:	15696.3478	Pseudo R-squ.:	0.0430			
Left threshold:	0	Right threshold:	none			
	coef	std err	z	P> z	[0.025	0.975]
sigma	4.4990	0.0771	58.3463	0.0000	4.3478	4.6501
const	7.8354	0.7135	10.9818	0.0000	6.4370	9.2338
rate_marriage	-1.5313	0.0734	-20.8514	0.0000	-1.6752	-1.3873
age	-0.1051	0.0248	-4.2393	0.0000	-0.1537	-0.0565
yrs_married	0.1282	0.0264	4.8576	0.0000	0.0765	0.1799
children	-0.0276	0.0771	-0.3576	0.7206	-0.1788	0.1236
religious	-0.9434	0.0849	-11.1077	0.0000	-1.1099	-0.7769
educ	-0.0857	0.0376	-2.2772	0.0228	-0.1594	-0.0119
occupation	0.3125	0.0819	3.8150	0.0001	0.1520	0.4731
occupation_husb	0.0143	0.0555	0.2578	0.7966	-0.0944	0.1230
Left-censored observations: 4313 (67.75%)						
Right-censored observations: 0 (0.00%)						
Uncensored observations: 2053 (32.25%)						
Optimiser converged: True						

Marginal effects

Because the response is left-censored at zero, the censored marginal effects are smaller in absolute value than the latent ones. The latent (uncensored) coefficients describe the propensity scale, while the censored marginal effects describe the *expected number* of affairs.

```
In [20]: ame_cens = model.ame(kind='censored').to_dataframe()
ame_lat = model.ame(kind='latent').to_dataframe()
pd.concat([
    ame_lat[['dy/dx']].rename(columns={'dy/dx': 'latent'}),
    ame_cens[['dy/dx', 'std err', 'P>|z|']].rename(columns={'dy/dx': 'censored E[Y]}'),
], axis=1).round(4)
```

	latent	censored E[Y]	std err	P> z
rate_marriage	-1.5313	-0.4326	0.0213	0.0000
age	-0.1051	-0.0297	0.0070	0.0000
yrs_married	0.1282	0.0362	0.0074	0.0000
children	-0.0276	-0.0078	0.0218	0.7206
religious	-0.9434	-0.2665	0.0243	0.0000
educ	-0.0857	-0.0242	0.0106	0.0228
occupation	0.3125	0.0883	0.0232	0.0001
occupation_husb	0.0143	0.0040	0.0157	0.7966

Sanity check vs. OLS

Naive OLS on the censored sample understates the slope coefficients (Greene 1981).
The Tobit MLE corrects this.

```
In [21]: X_int = sm.add_constant(X)
ols = sm.OLS(y, X_int).fit()
pd.DataFrame({
    'OLS coef':    ols.params.values,
    'Tobit coef': model.coef_,
}, index=ols.params.index).round(4)
```

```
Out[21]:
```

	OLS coef	Tobit coef
const	3.6235	7.8354
rate_marriage	-0.4205	-1.5313
age	-0.0146	-0.1051
yrs_married	-0.0160	0.1282
children	-0.0171	-0.0276
religious	-0.2437	-0.9434
educ	-0.0174	-0.0857
occupation	0.0658	0.3125
occupation_husb	0.0040	0.0143

2.3 Example 3 — Truncated Regression

```
In [22]: import numpy as np
import pandas as pd
from censtrunc import TruncatedRegression

rng = np.random.default_rng(7)
n_population = 8000
```

Simulate a truncated dataset

We draw a large population, then keep only observations with $0 < Y^* < 2.5$ — about 60% of the population survives the double-sided truncation in this DGP.

```
In [23]: X_pop = rng.normal(size=(n_population, 2))
beta_true = np.array([1.0, 0.5, -0.3])
y_star = beta_true[0] + X_pop @ beta_true[1:] + rng.normal(scale=1.0, size=n_population)
L, R = 0.0, 2.5
mask = (y_star > L) & (y_star < R)
X = X_pop[mask]
y = y_star[mask]
print(f'Observed after truncation: {len(y)} / {n_population}')
```

Observed after truncation: 5741 / 8000

Fit the truncated regression

```
In [24]: model = TruncatedRegression(left=L, right=R).fit(
          X, y, feature_names=['x1', 'x2'])
          print(model.summary())
```

```
=====
                        Truncated Regression Results
=====
Dep. Variable:          y                      No. Observations:      5741
Model:                  TruncatedRegression    Df Model:                  3
Method:                 MLE                   Df Residuals:             5737
Date:                  Sun, 31 May 2026        Log-Likelihood:          -4830.9612
Time:                  19:47:17                LL-Null:                 -5136.1192
AIC:                   9669.9225               LLR p-value:             2.962e-133
BIC:                   9696.5440               Pseudo R-squ.:          0.0594
Left truncation:       0                      Right truncation:        2.5
=====

```

	coef	std err	z	P> z	[0.025	0.975]
sigma	0.9944	0.0316	31.5070	0.0000	0.9326	1.0563
const	0.9946	0.0245	40.6756	0.0000	0.9467	1.0425
x1	0.4881	0.0335	14.5918	0.0000	0.4226	0.5537
x2	-0.3105	0.0268	-11.6040	0.0000	-0.3629	-0.2580

```
=====
Optimiser converged:    True
=====
```

Comparison: OLS on the truncated sample is biased

Naive OLS on the truncated sample treats the data as if it came from an unrestricted population — leading to biased slope estimates, typically shrunk toward zero.

```
In [25]: X_int = np.column_stack([np.ones(len(y)), X])
          ols_beta, *_ = np.linalg.lstsq(X_int, y, rcond=None)

          comparison = pd.DataFrame({
              'true':      beta_true,
              'OLS (biased)': ols_beta,
              'Truncated MLE': model.coef_,
          }, index=['const', 'x1', 'x2'])
          comparison['OLS bias'] = comparison['OLS (biased)'] - comparison['true']
          comparison['Truncated MLE bias'] = comparison['Truncated MLE'] - comparison['true']
          comparison.round(4)
```

```
Out [25]:
```

	true	OLS (biased)	Truncated MLE	OLS bias	Truncated MLE bias
const	1.0	1.1475	0.9946	0.1475	-0.0054
x1	0.5	0.1952	0.4881	-0.3048	-0.0119
x2	-0.3	-0.1241	-0.3105	0.1759	-0.0105

Predictions

For the truncated model only two prediction quantities are meaningful (there is no mass at the thresholds, so the region-probability columns do not apply):

- $h(\text{latent})$: $X'\hat{\beta}$ — the population mean

- `t` (truncated): $\mathbb{E}[Y|X, L < Y < R]$

```
In [26]: preds = model.predict(X[:6])          # default = 'ht' -> both columns
preds.insert(0, 'observed', y[:6])
preds.round(3)
```

```
Out[26]:
```

	observed	latent	truncated
0	1.590	0.902	1.104
1	0.317	1.137	1.202
2	1.514	1.081	1.178
3	1.376	0.947	1.122
4	1.085	0.764	1.047
5	0.074	0.481	0.936

2.4 Example 4 — Validation against OLS and R

```
In [27]: import numpy as np
import pandas as pd
import statsmodels.api as sm
from censtrunc import CensoredRegression, TruncatedRegression

rng = np.random.default_rng(0)
n = 1000
X = rng.normal(size=(n, 3))
y = 2.0 + 1.5*X[:,0] - 0.7*X[:,1] + 0.3*X[:,2] + rng.normal(scale=1.3, size=n)
```

No censoring $\Rightarrow$ OLS

We set `left=-1e6`, `right=1e6` so that no observation is censored. The censored log-likelihood then reduces to the Gaussian log-likelihood, whose maximiser is exactly the OLS coefficient vector.

```
In [28]: ols = sm.OLS(y, sm.add_constant(X)).fit()
cens = CensoredRegression(left=-1e6, right=1e6).fit(X, y)
trunc = TruncatedRegression(left=-1e6, right=1e6).fit(X, y)

pd.DataFrame({
    'OLS':          np.asarray(ols.params),
    'Censored MLE': cens.coef_,
    'Truncated MLE': trunc.coef_,
}, index=['const', 'x1', 'x2', 'x3']).round(6)
```

Out [28]:

	OLS	Censored MLE	Truncated MLE
const	2.046512	2.046482	2.046512
x1	1.434191	1.434221	1.434191
x2	-0.626598	-0.626603	-0.626598
x3	0.345980	0.345982	0.345980

The columns agree to several decimals. We can quantify the maximum discrepancy and confirm the scale parameter and log-likelihood match too (using the MLE scale

$$\hat{\sigma} = \sqrt{\text{SSR}/n}.$$

In [29]:

```
ols_beta = np.asarray(ols.params)
print(f'max |beta_censored - beta_OLS| = {np.max(np.abs(cens.coef_ - ols_beta)):.2e}')
print(f'max |beta_truncated - beta_OLS| = {np.max(np.abs(trunc.coef_ - ols_beta)):.2e}')
print(f'|sigma_censored - sqrt(SSR/n)| = {abs(cens.sigma_ - np.sqrt(ols.ssr/n)):.2e}')
print(f'|loglik_censored - loglik_OLS| = {abs(cens.llf_ - ols.llf_):.2e}')
```

```
max |beta_censored - beta_OLS| = 3.00e-05
max |beta_truncated - beta_OLS| = 4.44e-16
|sigma_censored - sqrt(SSR/n)| = 7.55e-07
|loglik_censored - loglik_OLS| = 5.90e-07
```

All discrepancies are at the level of the optimiser tolerance — the estimator behaves exactly as theory requires in the no-censoring limit, which is strong evidence the likelihood and its optimisation are implemented correctly.

With censoring: agreement with R, disagreement with OLS

The decisive test is a genuinely censored dataset. We compare four numbers per coefficient:

- **true** — the data-generating value;
- **censtrunc** — this package's MLE;
- **R survreg** — R's Gaussian `survreg`, the engine behind `AER::tobit`, computed by an entirely independent codebase;
- **OLS** — naive least squares, which is biased under censoring.

Two correct maximum-likelihood implementations must converge to the *same unique* optimum, so `censtrunc` and R should agree to optimiser tolerance — while both should differ from the biased OLS and sit close to the truth.

In [30]:

```
import shutil, subprocess, json, tempfile, os
from pathlib import Path

rng2 = np.random.default_rng(20260527)
nc = 4000
Xc = rng2.normal(size=(nc, 2))
y_star_c = 1.0 + 0.7*Xc[:, 0] - 0.4*Xc[:, 1] + rng2.normal(size=nc)
Lc, Rc = 0.0, 2.5
yc = np.clip(y_star_c, Lc, Rc)
```

```

cens = CensoredRegression(left=Lc, right=Rc).fit(Xc, yc)
ols_c = np.asarray(sm.OLS(yc, sm.add_constant(Xc)).fit().params)

# Run R's survreg via the bundled reference script, if R is available.
def _find_r_script():
    for c in [Path('tests/reference/fit_tobit.R'),
              Path('../tests/reference/fit_tobit.R')]:
        if c.exists():
            return c
    return None

r_beta = r_sigma = None
rscript, script_path = shutil.which('Rscript'), _find_r_script()
if rscript and script_path:
    fd, csvp = tempfile.mkstemp(suffix='.csv'); os.close(fd)
    np.savetxt(csvp, np.column_stack([yc, Xc]), delimiter=',', header='y,x1,x2', comments='')
    try:
        out = subprocess.run([rscript, str(script_path), csvp, str(Lc), str(Rc)],
                              capture_output=True, text=True, timeout=120, check=True)
        rt = json.loads(out.stdout)['tobit']
        r_beta = np.array([rt['coef']['(Intercept)'], rt['coef']['x1'], rt['coef']['x2']])
        r_sigma = float(rt['scale'])
    except Exception:
        r_beta = None
    finally:
        os.remove(csvp)

```

```

In [31]: if r_beta is not None:
        table = pd.DataFrame({
            'true': [1.0, 0.7, -0.4],
            'censtrunc': cens.coef_,
            'R survreg': r_beta,
            'OLS (biased)': ols_c,
        }, index=['const', 'x1', 'x2'])
        display(table.round(6))
        print(f'max |censtrunc - R| = {np.max(np.abs(cens.coef_ - r_beta)):.2e} (independent solvers, same optimum)')
        print(f'|sigma_censtrunc - sigma_R| = {abs(cens.sigma_ - r_sigma):.2e}')
        print(f'max |censtrunc - OLS| = {np.max(np.abs(cens.coef_ - ols_c)):.3f} (Tobit correction is real)')
    else:
        print('R not available at build time.')
        print('The live comparison runs in the test suite: tests/test_r_reference.py')
        print('censtrunc estimates:', cens.coef_.round(4))
        print('OLS (biased):', ols_c.round(4))

```

	true	censtrunc	R survreg	OLS (biased)
const	1.0	1.014284	1.014284	1.090364
x1	0.7	0.701331	0.701331	0.470177
x2	-0.4	-0.410105	-0.410105	-0.277838

```

max |censtrunc - R| = 3.05e-08 (independent solvers, same optimum)
|sigma_censtrunc - sigma_R| = 2.41e-08
max |censtrunc - OLS| = 0.231 (Tobit correction is real)

```

The `censtrunc` and `R` columns are identical to about six decimals, yet `max |censtrunc - R|` is a tiny *non-zero* number ($\sim 1\text{e-}8$): the two independent optimisers land on the same maximum without being bit-for-bit copies. Both recover the true coefficients, while OLS is visibly biased toward zero — exactly the censoring attenuation predicted by Greene (1981).

2.5 Example 5 — Visual: Truncated vs Censored

```
In [32]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from censtrunc import CensoredRegression, TruncatedRegression
from censtrunc._means import value_for_kind
from censtrunc._utils import _prepare_design_matrix

rng = np.random.default_rng(2026)
n = 250
x = rng.uniform(-10, 10, size=n)
sigma_true = 2.0
y_star = 1.0 * x + rng.normal(scale=sigma_true, size=n)
L, R = -5.0, 5.0
```

Truncated dataset: observations with $y \notin (L, R)$ are dropped entirely. **Censored dataset:** they are clipped to the nearest threshold. The latent linear function $y = x$ is the same in both.

```
In [33]: mask = (y_star > L) & (y_star < R)
x_t, y_t = x[mask], y_star[mask]
x_c, y_c = x.copy(), np.clip(y_star, L, R)

mt = TruncatedRegression(left=L, right=R).fit(x_t.reshape(-1, 1), y_t)
mc = CensoredRegression(left=L, right=R).fit(x_c.reshape(-1, 1), y_c)
print(f'truncated:  n={len(y_t)}, beta_hat = {mt.coef_.round(3)}')
print(f'censored:   n={len(y_c)}, beta_hat = {mc.coef_.round(3)}')
```

```
truncated:  n=146, beta_hat = [-0.003  0.946]
censored:   n=250, beta_hat = [0.059 0.939]
```

Now we build a prediction band from asymptotic uncertainty. We draw 200 parameter vectors from the asymptotic normal distribution of $\hat{\theta}$, compute the corresponding conditional mean curve (truncated on the left, censored on the right), and plot them with low opacity. The pile-up at the censoring thresholds is shown in red on the right.

```
In [34]: def prediction_band(model, x_grid, kind, n_draws=200, seed=0):
    rng = np.random.default_rng(seed)
    samples = rng.multivariate_normal(model.params_, model.cov_params_, size=n_draws)
    samples[samples[:, 0] > 0] # keep only feasible sigma > 0
    Xg, _ = _prepare_design_matrix(x_grid.reshape(-1, 1), fit_intercept=True)
    curves = np.empty((samples.shape[0], x_grid.size))
    for i, theta in enumerate(samples):
        sigma, beta = theta[0], theta[1:]
        curves[i] = value_for_kind(
            beta, sigma, Xg,
            model._left, model._right, model._has_left, model._has_right, kind,
        )
    return curves

x_grid = np.linspace(-10, 10, 300)
curves_t = prediction_band(mt, x_grid, kind='truncated')
curves_c = prediction_band(mc, x_grid, kind='censored')
```

```
In [35]: fig, axes = plt.subplots(1, 2, figsize=(13, 5.5), sharey=True)
for ax in axes:
    ax.set_facecolor('#eaeaf2')
    ax.grid(True, color='white', linewidth=1)
    ax.axhline(L, color='crimson', linestyle='--', linewidth=1.2)
    ax.axhline(R, color='crimson', linestyle='--', linewidth=1.2)
    ax.set_xlim(-10, 10); ax.set_ylim(-10, 10)
    ax.set_xlabel('x'); ax.set_ylabel('y')
    ax.plot(x_grid, x_grid, color='black', linewidth=2.5, label='True')
```



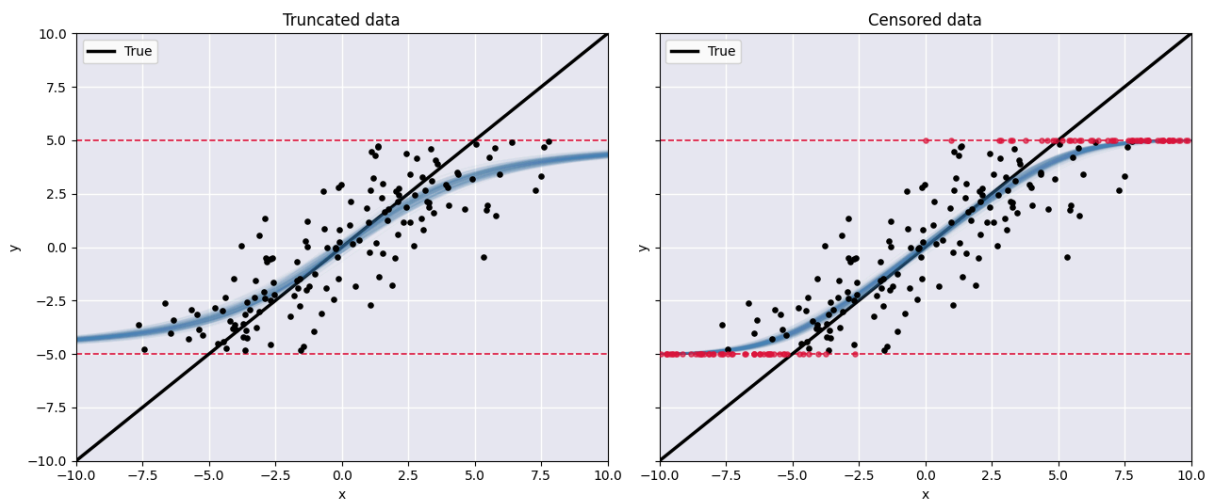
```

# Left panel: truncated
ax = axes[0]
ax.set_title('Truncated data')
for c in curves_t:
    ax.plot(x_grid, c, color='steelblue', alpha=0.04, linewidth=1)
ax.scatter(x_t, y_t, c='black', s=14, zorder=5)
ax.legend(loc='upper left')

# Right panel: censored
ax = axes[1]
ax.set_title('Censored data')
for c in curves_c:
    ax.plot(x_grid, c, color='steelblue', alpha=0.04, linewidth=1)
is_at_bound = (y_c == L) | (y_c == R)
ax.scatter(x_c[~is_at_bound], y_c[~is_at_bound], c='black', s=14, zorder=5)
ax.scatter(x_c[is_at_bound], y_c[is_at_bound], c='crimson', s=14, zorder=5, alpha=0.7)
ax.legend(loc='upper left')

plt.tight_layout(); plt.show()

```



What to look for.

- The black line is the true linear relation $y^* = x$.
- The blue band is the model's MLE prediction with asymptotic uncertainty; on the **left** it bends inside (L, R) because the truncated mean $\mathbb{E}[Y \mid X, L < Y < R]$ is shrunk toward the interval centre; on the **right** it flattens near each threshold because the censored mean $\mathbb{E}[Y \mid X]$ mixes the latent linear part with the threshold mass.
- Red dots on the right panel are the censored pile-ups at $y = L$ and $y = R$. The truncated dataset has no such pile-ups because those observations are absent entirely.
- Both fitted curves are close to the true line in the interior, where the data are most informative; both lose precision near and beyond the thresholds, but neither shows the strong attenuation toward zero that OLS would suffer.

2.6 Example 6 — Heckman Selection (Heckit)

```
In [36]: import numpy as np
import pandas as pd
import statsmodels.api as sm
from censtrunc import HeckitRegression

rng = np.random.default_rng(42)
n = 4000
# Shared regressor (in both equations); two exclusive regressors
shared = rng.normal(size=n)
x_only = rng.normal(size=n)      # in the outcome eq. only
z_only = rng.normal(size=n)      # in the selection eq. only (exclusion)

beta_true = np.array([1.0, 0.5, -0.3]) # const, shared, x_only
gamma_true = np.array([0.0, 0.3, 0.6]) # const, shared, z_only
rho_true, sigma_true = 0.6, 1.0

Sigma = np.array([[sigma_true**2, rho_true*sigma_true],
                  [rho_true*sigma_true, 1.0]])
errs = rng.multivariate_normal([0.0, 0.0], Sigma, size=n)
e, u = errs[:, 0], errs[:, 1]

X = np.column_stack([shared, x_only])
Z = np.column_stack([shared, z_only])
S = (gamma_true[0] + Z @ gamma_true[1:] + u > 0).astype(int)
y_full = beta_true[0] + X @ beta_true[1:] + e
y = np.where(S == 1, y_full, np.nan)
print(f'Selected (S=1): {S.sum()} / {n} ({S.mean():.1%})')
```

Selected (S=1): 2022 / 4000 (50.5%)

Why OLS on the selected sample is biased

Because u and e are correlated ($\rho > 0$), individuals with high e tend to have high u and therefore are more likely to be selected. The conditional mean for the selected sample is

$$\mathbb{E}[Y \mid X, S = 1] = X'\beta + \rho\sigma \lambda(Z'\gamma),$$

with $\lambda(\cdot)$ the inverse Mills ratio. Omitting $\lambda(Z'\gamma)$ from the regression — what naive OLS does — produces omitted-variable bias on β .

```
In [37]: sel = ~np.isnan(y)
X_full = sm.add_constant(X)
ols = sm.OLS(y[sel], X_full[sel]).fit()
print('OLS on selected subsample (biased):')
print(np.asarray(ols.params).round(4))
print('truth:', beta_true)
```

```
OLS on selected subsample (biased):
[ 1.3651  0.4366 -0.3037]
truth: [ 1.    0.5 -0.3]
```

Two-step Heckit

Heckman's two-step recipe:

1. Probit of S on Z to obtain $\hat{\gamma}$.
2. Compute the inverse Mills ratio $\hat{\lambda}_i = \phi(Z_i'\hat{\gamma})/\Phi(Z_i'\hat{\gamma})$ for selected observations.
3. OLS of y_i on $(X_i, \hat{\lambda}_i)$ for selected i . The coefficients are $\hat{\beta}$ and $\hat{\rho}\hat{\sigma}$.

```
In [38]: m_two = HeckitRegression(method='twostep').fit(y, X, Z)
print(m_two.summary())
```

```

=====
Heckman Selection Regression
=====
Method:                twostep                No. Observations:    4000
Selected (S=1):        2022                  Censored (S=0):      1978
sigma_e:               0.952857              rho:                 0.531381
=====

Outcome equation (Y* = X * beta + e)
-----
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const          1.0267      0.0491     20.9278     0.0000      0.9306      1.1229
x1              0.4903      0.0223     21.9471     0.0000      0.4465      0.5341
x2             -0.3010      0.0209    -14.4109     0.0000     -0.3419     -0.2601
-----

Selection equation (S* = Z * gamma + u, var(u) = 1)
-----
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const          0.0333      0.0213      1.5611     0.1185     -0.0085      0.0751
x1              0.2585      0.0218     11.8357     0.0000      0.2157      0.3013
x2              0.6260      0.0242     25.8990     0.0000      0.5786      0.6734
=====
Note: two-step second-stage standard errors are naive; call .bootstrap(y, X, Z) for proper Heckman-correc
=====

```

Joint MLE Heckit

Maximise the full log-likelihood (Hansen 2022, §27.10):

$$\ell = \sum_{S_i=0} \log[1 - \Phi(Z_i'\gamma)] + \sum_{S_i=1} \left\{ \log \Phi \left(\frac{Z_i'\gamma + (\rho/\sigma)(Y_i - X_i'\beta)}{\sqrt{1 - \rho^2}} \right) - \frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2} \right\}$$

Asymptotically efficient, somewhat slower than the two-step.

```
In [39]: m_ml = HeckitRegression(method='mle').fit(y, X, Z)
print(m_ml.summary())
```

```

=====
Heckman Selection Regression
=====
Method:                mle                No. Observations:    4000
Selected (S=1):        2022                  Censored (S=0):      1978
sigma_e:               0.958607              rho:                 0.552343
Log-Likelihood:        -4924.8374
=====

Outcome equation (Y* = X * beta + e)
-----
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const          1.0108      0.0446     22.6544     0.0000      0.9234      1.0983
x1              0.4925      0.0214     23.0340     0.0000      0.4506      0.5344
x2             -0.3017      0.0192    -15.7382     0.0000     -0.3393     -0.2642
-----

Selection equation (S* = Z * gamma + u, var(u) = 1)
-----
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const          0.0345      0.0213      1.6183     0.1056     -0.0073      0.0763
x1              0.2598      0.0218     11.9411     0.0000      0.2171      0.3024
x2              0.6242      0.0241     25.8950     0.0000      0.5769      0.6714
=====

```

Side-by-side comparison

All three estimators on the same data. Heckit's two-step and MLE recover β_{shared} much closer to the truth than OLS, and they recover ρ and σ as well.

```
In [40]: comparison = pd.DataFrame({
    'true': beta_true,
    'OLS (biased)': np.asarray(ols.params),
    'Heckit 2step': m_two.coef_,
    'Heckit MLE': m_ml.coef_,
    }, index=['const', 'shared', 'x_only'])
comparison['OLS error'] = comparison['OLS (biased)'] - comparison['true']
comparison['2step error'] = comparison['Heckit 2step'] - comparison['true']
comparison['MLE error'] = comparison['Heckit MLE'] - comparison['true']
comparison.round(4)
```

```
Out[40]:
```

	true	OLS (biased)	Heckit 2step	Heckit MLE	OLS error	2step error	MLE error
const	1.0	1.3651	1.0267	1.0108	0.3651	0.0267	0.0108
shared	0.5	0.4366	0.4903	0.4925	-0.0634	-0.0097	-0.0075
x_only	-0.3	-0.3037	-0.3010	-0.3017	-0.0037	-0.0010	-0.0017

```
In [41]: pd.DataFrame({
    'true': [rho_true, sigma_true],
    'two-step': [m_two.rho_, m_two.sigma_],
    'MLE': [m_ml.rho_, m_ml.sigma_],
    }, index=['rho', 'sigma']).round(4)
```

```
Out[41]:
```

	true	two-step	MLE
rho	0.6	0.5314	0.5523
sigma	1.0	0.9529	0.9586

Three kinds of prediction

- `kind='selection_prob'` — $P(S = 1 \mid Z) = \Phi(Z'\hat{\gamma})$,
- `kind='outcome'` — $\mathbb{E}[Y^* \mid X] = X'\hat{\beta}$ (unconditional),
- `kind='conditional'` — $\mathbb{E}[Y \mid X, Z, S = 1] = X'\hat{\beta} + \hat{\rho}\hat{\sigma}\lambda(Z'\hat{\gamma})$.

```
In [42]: preds = pd.DataFrame({
    'selection_prob': m_ml.predict(Z=Z[:6], kind='selection_prob'),
    'outcome': m_ml.predict(X=X[:6], kind='outcome'),
    'conditional': m_ml.predict(X=X[:6], Z=Z[:6], kind='conditional'),
    'observed_y': y[:6],
    'selected': S[:6],
    })
preds.round(3)
```

Out [42]:

	selection_prob	outcome	conditional	observed_y	selected
0	0.626	1.084	1.405	1.327	1
1	0.704	0.229	0.488	1.942	1
2	0.839	1.298	1.453	1.469	1
3	0.499	0.798	1.222	1.666	1
4	0.363	-0.381	0.166	-0.354	1
5	0.071	0.462	1.474	NaN	0

Same fit via a patsy formula

Instead of building `y`, `X`, `Z` by hand, every model class accepts a `from_formula(...)` constructor in the statsmodels style. For Heckit you pass *two* formulas — outcome and selection — sharing a single dataframe:

```
In [43]: df = pd.DataFrame({
    'shared': shared, 'x_only': x_only, 'z_only': z_only,
    'inlf': S, 'lwage': y_full,
})
m_form = HeckitRegression.from_formula(
    outcome = 'lwage ~ 1 + shared + x_only',
    selection = 'inlf ~ 1 + shared + z_only',
    data=df, method='mle',
).fit()
print(pd.DataFrame({
    'beta (explicit)': m_ml.coef_,
    'beta (formula)': m_form.coef_,
}, index=['const', 'shared', 'x_only']).round(6))
```

	beta (explicit)	beta (formula)
const	1.010834	1.010834
shared	0.492478	0.492478
x_only	-0.301747	-0.301747

Bootstrap standard errors

Two-step second-stage standard errors are naive: they ignore the variability of $\hat{\gamma}$ from the probit step. A paired bootstrap gives proper SEs. The MLE Hessian-based SEs are already correct.

```
In [44]: boot = m_two.bootstrap(y, X, Z, n_boot=80, seed=0)
pd.DataFrame({
    'beta': m_two.coef_,
    'naive SE': m_two.bse_,
    'bootstrap SE': boot['beta_se'],
}, index=['const', 'shared', 'x_only']).round(4)
```

Out[44]:

	beta	naive SE	bootstrap SE
const	1.0267	0.0491	0.0479
shared	0.4903	0.0223	0.0214
x_only	-0.3010	0.0209	0.0205